

Phase 2 — Agent 核心引擎

SPEC-004 | Status: 设计中 | 依赖: SPEC-003

目标

实现 Agent 核心引擎：LLM 调度、Skill 接口与注册、Chat/Agent 服务、安全审计横切层、上下文管理、Vault 密钥、子 Agent 编排。

异步任务队列、Worker Pool、Scheduler 调度器已移入 **SPEC-009**（任务队列与调度基础设施）。
具体 Skill 实现见 **SPEC-008**（Skill 实现层）。

前置依赖检查

前置 Spec	状态	备注
SPEC-002	✅/❌	CI Pipeline 就绪
SPEC-003	✅/❌	基础设施（MongoDB/Redis/Milvus/SeaweedFS）可用

背景

Roadmap Phase 2 (Week 3-4)，覆盖 P2-01 ~ P2-04（Engine + Router + Skill 接口 + Loader），P2-09 ~ P2-11c（Session/Chat/Prompt/Vault/Context），P4-05 ~ P4-06（子 Agent 编排），P4-10 ~ P4-11（安全审计 + 熔断器）。

详细设计

1. Agent Engine (ADK)

- ADK Agent Engine 初始化与配置

- LLM Router: 多模型切换 (GPT-4o/Claude/Gemini)、参数配置 (temperature/max_tokens/top_p)
- **模型配置优先级**: 数据库配置 (后台设置) > 环境变量默认值
- 环境变量 `LLM_MODEL`、`LLM_BASE_URL`、`LLM_API_KEY` 作为系统默认值
- MongoDB `model_configs` 集合中的配置覆盖环境变量
- 后台修改后立即生效 (Watcher 监听或配置热加载)
- 模型配置持久化: MongoDB `model_configs` 集合
- 仅 system_admin 可修改模型配置

2. Skill 接口与注册中心

- Skill 接口定义:

```
type Skill interface {
    Name() string

    Description() string

    Parameters() []Parameter

    Execute(ctx SkillContext, params map[string]any) (any, error)

    Permissions() []string

    RateLimit() RateLimitConfig
}
```

- Skill 自动加载器: 扫描 `skills/` 目录, 按子目录名注册
- SkillContext 自动注入: SessionID / UserID / TaskID, Skill 实现层不可覆盖
- 具体 Skill 实现 (sql_executor, stats_engine, knowledge_search 等) 见 SPEC-008

3. Chat Service

- SSE 流式响应: 每个 token 增量推送
- Session Manager: 创建 `POST /sessions`、续期、TTL 过期自动清理
- 快捷提示词 CRUD (MongoDB `prompts` 集合) + 按角色展示
- Prompt Enhancement Service (无状态): 用户输入 → LLM 生成 3 个增强版本建议

4. Agent Service (统一入口)

- Chat 实时模式: 同步调用 LLM → SSE 流式返回
- Agent 同步模式: 串行执行 Skill 链 → 返回最终结果
- Agent 异步模式: 创建 AgentTask → 入队 (见 SPEC-009) → 立即返回 `task_id`
- AgentTask 数据模型: `{task_id, session_id, user_id, skill_chain, status, created_at}`
- 注意: Worker Pool、Task Queue (Redis Stream)、任务取消/进度/通知均见 SPEC-009

5. 安全审计层 (Security Audit Layer)

安全审计是 Agent **核心横切能力**——所有请求链路上的入口与出口均需经过校验。

- **Input Sanitization** (LLM 调用前):
 - Regex + Keyword Filter: 拦截 SQL 注入、XSS、敏感词
 - 白名单校验: 仅允许预定义的数据分析指令
- **Output Sanitization** (LLM 返回后):
 - 敏感信息脱敏: 手机号 `138****1234`、身份证 `320*****1234`、API Key 掩码
- **Tool Call 审计** (Skill 执行前):
 - 拦截高风险 Skill 调用 (如 `workspace_exec` 写 `/etc/` 等敏感路径)
 - 记录所有 Skill 调用链到 `security_alerts` 集合
- **Hot-reload 安全规则**: Redis pub/sub 推送规则更新, 无需重启服务

6. 熔断器 (Circuit Breaker)

- 连续失败 N 次 → 打开 (Open), 拒绝后续请求, 返回 HTTP 503
- 冷却时间 T 秒后 → 半开 (Half-Open), 允许 1 个探测请求
- 探测成功 → 关闭 (Closed), 恢复正常
- 探测失败 → 回到 Open, 重置冷却计时
- **按 Skill 粒度独立熔断**: 避免单个 Skill (如 SQL Executor 数据库断开) 拖垮整个 Agent

7. 上下文窗口管理

- tiktoken-go token 计数 (按模型动态选择 encoder)
- 阈值 = `max_context_tokens × 50%` (从 `model_configs` 读取)
- 触发策略:
 - LLM 摘要压缩: 用轻量模型 (gpt-4o-mini) 生成 ~100 token 历史摘要
 - KB 检索结果截断: top-5 × 800 tokens
 - 长报告分段生成 + 合并

8. Vault 密钥管理

- AES-256-GCM 加密存储: API Key / DB 密码 / 第三方 Secret
- 密钥轮转: 定时轮转 + 手动触发 + 旧密钥保留 7 天 (解密历史数据)
- 访问控制: 仅 Agent Engine / Agent Service 可解密

9. 子 Agent 编排 (Phase 4 实现)

- Sub-Agent 编排基础 (A2A 协议): 主 Agent 调度子 Agent 执行分析子任务
- 子 Agent 会话生命周期:

- 创建：主 Agent 发起 → 分配独立 session_id
- 注入上下文：父 Agent 的 KB 检索结果、SQL 查询结果等
- 结果回传：子 Agent 完成后 → 结构化回传主 Agent
- 超时回收：子 Agent 超时 → 强制终止 + 返回部分结果
- 子 Agent 隔离：独立 Skill 权限集（如仅允许 `sql_executor` + `stats_engine`），不可越权

可行性分析

检查项	结论
是否需要新 DB 集合	Yes (sessions, model_configs, vault_secrets, prompts, prompt_enhancements, security_alerts, token_consumptions)
是否影响现有 API	No
性能影响	Input/Output 过滤 < 2ms, 熔断器判断 O(1)
是否需要新增 Skill	No (Skill 实现见 SPEC-008)
是否需要 E2E 测试	Chat → SSE 流式返回 UI 用例；熔断器打开→拒绝→恢复 UI 用例

相关文件

文件	角色	变更幅度
<code>internal/domain/agent/engine.go</code>	Agent Engine 核心	新建
<code>internal/domain/skill/registry.go</code>	Skill 注册中心	新建
<code>internal/domain/skill/context.go</code>	SkillContext	新建
<code>internal/domain/security/audit.go</code>	安全审计引擎	新建
<code>internal/domain/security/circuit_breaker.go</code>	熔断器	新建
<code>internal/domain/security/rules.yaml</code>	默认安全规则	新建
<code>internal/service/chat/</code>	Chat Service (SSE)	新建
<code>internal/service/agent/</code>	Agent Service (统一入口)	新建

验证标准

- 1. Chat 模式：发送消息 → SSE 流式返回 → Tool Call 调用链可见
- 2. Agent 异步：创建任务 → 返回 task_id → 结果持久化（见 SPEC-009 验证）
- 3. SQL 注入输入（如 `' ; DROP TABLE users; --`）→ Input Sanitization 拦截并记录 alert
- 4. 输出含手机号 `13812345678` → Output Sanitization 脱敏为 `138****5678`
- 5. 连续 5 次 Skill 执行失败 → 该 Skill 熔断器打开 → 返回 503
- 6. 冷却 30s 后探测请求成功 → 熔断器关闭 → 恢复正常
- 7. Redis pub/sub 推送新规则 → 无需重启，即时生效
- 8. Skill 自动加载：新增 Skill 目录 → 重启后自动注册
- 9. 上下文窗口：超 50% 阈值 → 自动触发摘要压缩